这是一道要求实现 **扩展卢卡斯定理 (Extended Lucas Theorem)** 的经典模板题。

与普通的卢卡斯定理（要求 p 是质数）不同，由于题目明确指出 p 不一定是质数，我们需要使用扩展卢卡斯定理。

算法思路

求组合数 $\mathrm{C}_n^m \bmod p$ 且 p 不是质数时，我们的解题步骤如下：

1. **质因数分解：** 将 p 分解为若干个素数幂的乘积，即 $p = p_1^{k_1} p_2^{k_2} \cdots p_r^{k_r}$。
2. **分别求解并用CRT合并：** 对每一个 $p_i^{k_i}$，分别计算出 $\mathrm{C}_n^m \bmod p_i^{k_i}$ 的值，最后运用**中国剩余定理（CRT）**合并这些结果，即可得到最终模 p 的答案。
3. **计算 $\mathrm{C}_n^m \bmod p_i^{k_i}$：**
   组合数定义为 $\frac{n!}{m!(n-m)!}$。在模 $p_i^{k_i}$ 意义下，分母的阶乘可能与 p_i 不互素，因此没有逆元。我们可以把 $n!$ 提取出所有的 p_i 因子，记剩余互质的部分为 $f(n)$。
   则 $\mathrm{C}_n^m = p_i^{v_{p_i}(n!)-v_{p_i}(m!)-v_{p_i}((n-m)!)} \times f(n) \times f(m)^{-1} \times f(n-m)^{-1} \pmod{p_i^{k_i}}$。
4. **求 $f(n)$：** 我们利用以 $p_i^{k_i}$ 为周期的性质，可以预处理出前 $p_i^{k_i}$ 个数中不包含因子 p_i 的乘积的前缀数组。剩余的可以直接用快速幂处理整个周期部分，不完整的尾段查表解决。然后递归计算即可。

C++ 代码实现

```cpp
#include <iostream>

using namespace std;

typedef long long ll;

// 扩展欧几里得算法，求 ax + by = gcd(a, b)
void exgcd(ll a, ll b, ll &x, ll &y) {
    if (!b) {
        x = 1;
        y = 0;
        return;
    }
    exgcd(b, a % b, y, x);
    y -= a / b * x;
}

// 求 a 在模 p 意义下的乘法逆元 (保证 a 和 p 互素)
ll inv(ll a, ll p) {
    ll x, y;
    exgcd(a, p, x, y);
    return (x % p + p) % p;
}

// 快速幂计算 a^b mod p
ll qpow(ll a, ll b, ll p) {
    ll res = 1;
```

```
    a %= p;
    while (b) {
        if (b & 1) res = res * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return res;
}

// 根据题目的 p <= 10^6，单个 pk 不会超过 10^6，预处理数组大小设为 1000005 是安全的
const int MAXP = 1000005;
ll pre[MAXP];

// 递归计算 n! 中与 pi 互素的部分模 pk 的结果
ll calc_fac(ll n, ll pi, ll pk) {
    if (!n) return 1;
    // pre[pk] 也就是 1 到 pk 中与 pi 互素的数的乘积
    // n / pk 为完整循环节的个数；n % pk 为末尾剩余不够一个循环节的部分
    ll res = qpow(pre[pk], n / pk, pk) * pre[n % pk] % pk;
    return res * calc_fac(n / pi, pi, pk) % pk;
}

// 计算组合数 C(n, m) mod pk，其中 pk = pi^e
ll C(ll n, ll m, ll pi, ll pk) {
    if (n < m) return 0;

    // 勒让德定理：计算 n! / (m! * (n-m)!) 包含的 pi 的确切次幂数
    ll k = 0;
    for (ll i = n; i; i /= pi) k += i / pi;
    for (ll i = m; i; i /= pi) k -= i / pi;
    for (ll i = n - m; i; i /= pi) k -= i / pi;

    // 预处理 1 到 pk 中与 pi 互素的数的乘积前缀
    pre[0] = 1;
    for (ll i = 1; i <= pk; ++i) {
        pre[i] = pre[i - 1];
        if (i % pi) pre[i] = pre[i] * i % pk;
    }

    ll a = calc_fac(n, pi, pk);
    ll b = calc_fac(m, pi, pk);
    ll c = calc_fac(n - m, pi, pk);

    // 组合数公式，排除 pi 因子后的部分应用乘法逆元计算，再乘回因子的差值对应幂次
    return a * inv(b, pk) % pk * inv(c, pk) % pk * qpow(pi, k, pk) % pk;
}

// 中国剩余定理（CRT）的一个方程的合并
ll CRT(ll b, ll pk, ll p) {
    ll p_div_pk = p / pk;
    return b * p_div_pk % p * inv(p_div_pk, pk) % p;
}
```

```
// 扩展卢卡斯定理主函数
ll exLucas(ll n, ll m, ll p) {
    ll res = 0;
    ll tmp = p;

    // 对 p 进行质因数分解
    for (ll i = 2; i * i <= tmp; ++i) {
        if (tmp % i == 0) {
            ll pk = 1;
            while (tmp % i == 0) {
                pk *= i;
                tmp /= i;
            }
            // 利用 CRT 逐个累加每个素数幂模块提供的贡献
            res = (res + CRT(C(n, m, i, pk), pk, p)) % p;
        }
    }
    // 扫尾，如果 tmp > 1，说明 tmp 本身是一个大的质数因子
    if (tmp > 1) {
        res = (res + CRT(C(n, m, tmp, tmp), tmp, p)) % p;
    }
    return res;
}

int main() {
    // 优化输入输出流
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    ll n, m, p;
    if (cin >> n >> m >> p) {
        cout << exLucas(n, m, p) << "\n";
    }

    return 0;
}
```

这是一份常用的 **卢卡斯定理 (Lucas' Theorem)** C++ 模板。

卢卡斯定理主要用于求解 **大组合数取模** 的问题：即计算 $C(n, m) \pmod p$，其中 n, m 可以非常大（例如 10^{18}），而模数 p 较小（通常 $p \le 10^5$），且 p **必须是素数。**

根据不同的应用场景（单次查询 vs 多次查询），这里提供两种最常用的写法。

模板一：单次查询（无需预处理）

如果只有少量询问，或者每次询问的模数 p 都在变化，可以直接暴力计算较小规模的组合数。
时间复杂度：单次查询约为 $O(p \log_p n \log p)$。

```
#include <iostream>
```

```cpp
using namespace std;
typedef long long ll;

// 快速幂，用于求逆元
ll qpow(ll a, ll b, ll p) {
    ll res = 1;
    a %= p;
    while (b > 0) {
        if (b & 1) res = res * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return res;
}

// 费马小定理求逆元（要求 p 是素数）
ll inv(ll a, ll p) {
    return qpow(a, p - 2, p);
}

// 计算普通的组合数 C(n, m) % p，要求 n, m < p
ll C(ll n, ll m, ll p) {
    if (m < 0 || m > n) return 0;
    if (m == 0 || m == n) return 1;
    if (m > n - m) m = n - m; // 组合数性质优化，减小计算量

    ll up = 1, down = 1;
    for (ll i = 1; i <= m; i++) {
        up = up * (n - i + 1) % p;
        down = down * i % p;
    }
    return up * inv(down, p) % p;
}

// 卢卡斯定理递归计算 C(n, m) % p
ll lucas(ll n, ll m, ll p) {
    if (m == 0) return 1;
    if (n < m) return 0; // 边界条件
    // Lucas 定理：C(n, m) % p = C(n % p, m % p) * C(n / p, m / p) % p
    return C(n % p, m % p, p) * lucas(n / p, m / p, p) % p;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    ll n, m, p;
    // 测试：计算 C(n, m) % p
    while (cin >> n >> m >> p) {
        cout << lucas(n, m, p) << "\n";
    }
    return 0;
```

```
}
```

模板二：多次查询（预处理阶乘）

如果模数 p 是固定的（或者查询次数非常多），且 $p \le 10^5$，我们通常会在 $O(p)$ 时间内先预处理出阶乘及其逆元，这样每次调用 `C(n, m)` 的时间就降到了 $O(1)$。

时间复杂度：预处理 $O(p)$，单次查询 $O(\log_p n)$。

```cpp
#include <iostream>
#include <vector>

using namespace std;
typedef long long ll;

const int MAXP = 100005; // 根据题目最大的 p 设定，注意 p 不能超过 10^5 级别，否则会超内存
ll fact[MAXP], invFact[MAXP];

// 快速幂
ll qpow(ll a, ll b, ll p) {
    ll res = 1;
    a %= p;
    while (b) {
        if (b & 1) res = res * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return res;
}

// 预处理阶乘及其逆元
void init(int p) {
    fact[0] = 1;
    invFact[0] = 1;
    for (int i = 1; i < p; i++) {
        fact[i] = fact[i - 1] * i % p;
    }
    // 费马小定理求 p-1 的阶乘逆元
    invFact[p - 1] = qpow(fact[p - 1], p - 2, p);
    for (int i = p - 2; i >= 1; i--) {
        invFact[i] = invFact[i + 1] * (i + 1) % p;
    }
}

// O(1) 求组合数 C(n, m) % p
ll C(ll n, ll m, ll p) {
    if (m < 0 || m > n) return 0;
    if (m == 0 || m == n) return 1;
    return fact[n] * invFact[m] % p * invFact[n - m] % p;
}
```

```cpp
// 卢卡斯定理
ll lucas(ll n, ll m, ll p) {
    if (m == 0) return 1;
    if (n < m) return 0;
    return C(n % p, m % p, p) * lucas(n / p, m / p, p) % p;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;
    // 假设本题所有的查询都是针对同一个固定的 p
    // 如果 p 不同，需要每次查询前调用 init(p)，前提是 p 不能过大
    ll p = 100003;
    init(p);

    while (t--) {
        ll n, m;
        cin >> n >> m;
        cout << lucas(n, m, p) << "\n";
    }
    return 0;
}
```

总结与排错提示：

1. p **不是素数怎么办**：Lucas 定理严格要求 p 必须是素数。如果 p 不是素数，需要使用 **扩展卢卡斯定理 (exLucas)**，它需要用到中国剩余定理 (CRT)。
2. `n < m` **的情况**：在某些子问题分解中会出现 `n % p < m % p` 的情况，此时直接返回 0 是正确的（这是组合数定义域的自然推广）。代码中已经包含此判断。
3. **当模数非常大时**（例如 $p = 10^9 + 7$ 或 998244353），如果此时 n 和 m 也在 10^5 级别，其实完全没必要使用 Lucas，直接使用模板二里的 `C(n, m, p)` 函数就可以了。卢卡斯定理仅针对 p **较小，而** n, m **很大** 的场景。